

Rapport Technique de Projet

Système de Gestion du Trafic de Fret Portuaire
(Port Autonome de Douala)

Groupe de 10 Étudiants (TP308)

July 4, 2026

Présentation Générale du Projet

L'application développée est un système de gestion portuaire destiné au **Port Autonome de Douala**. L'objectif est de numériser le suivi des marchandises arrivant par bateau. L'application gère deux types de cargaisons : les conteneurs standards et les conteneurs réfrigérés, ces derniers nécessitant une surveillance thermique stricte.

Techniquement, le système est une application Desktop en **Java Swing** qui intègre :

- Une modélisation **Orientée Objet** robuste (héritage, classes abstraites).
- Une interface graphique fluide et réactive avec **FlatLaf** (support mode clair/sombre).
- Un simulateur **Multithread** d'évolution des températures.
- Une **Persistance binaire** des données pour la sauvegarde de l'état.

1 Organisation et Répartition des Tâches

Le groupe a fonctionné comme une équipe technique agile, divisée en 4 pôles spécialisés.

1.1 1. Équipe Core & Métier

Membres : Bahebeg, Olivia, Ali

Mission : Modélisation de la POO, classes abstraites, héritage et algorithmes métier.

L'équipe a conçu la structure de base des objets métier. Le fichier central est la classe abstraite `Marchandise.java` qui encapsule les attributs communs (ID, poids, provenance, statut). Les sous-classes `ConteneurStandard.java` et `ConteneurRefrigere.java` en héritent. L'équipe a implémenté le **polymorphisme** sur le calcul des taxes portuaires via la méthode abstraite :

```
// Dans Marchandise.java
public abstract double calculerTaxe();

// Dans ConteneurRefrigere.java (taxe = 5% poids + 2% surcharge energetique)
@Override
public double calculerTaxe() {
    return getPoids() * 0.05 + getPoids() * 0.02;
}
```

1.2 2. Équipe Persistance & Données

Membres : Ali Fadel, Abdel

Mission : Écriture des modules d'accès aux données (Fichiers sérialisés).

Toute la logique d'enregistrement réside dans `GestionDonnees.java`. Cette équipe a mis en place la sérialisation binaire permettant de sauvegarder la liste complète (collection `List<Marchandise>`) dans un fichier `port_douala.ser`. Pour assurer la sécurité lors du multithreading, l'équipe a intégré un système de **verrous** (`synchronized (verrou)`).

```
// Dans GestionDonnees.java
public void sauvegarder(String chemin) throws IOException {
    synchronized (verrou) {
        try (ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream(chemin))) {
            oos.writeObject(new ArrayList<>(cargaisons));
        }
    }
}
```

1.3 3. Équipe IHM Swing

Membres : Alamine, Bayi, Amayen

Mission : Conception des fenêtres, agencement des composants (Layouts), événements.

L'interface graphique est orchestrée par `FenetrePrincipale.java`. L'équipe a géré l'affichage de la `JTable` avec un rendu coloré personnalisé dans `RenduTableauCargaison.java` (badges de statut arrondis). Les formulaires de saisie (`DialogueCargaison.java`) utilisent un `GridBagLayout` enveloppé dans un composant `Scrollable` pour éviter le rognage sur petit écran.

Un des éléments majeurs de l'IHM est le tableau de bord qui affiche des statistiques et le composant `GraphiqueTemperature.java`. **Ce graphique dynamique** utilise `Graphics2D` pour dessiner l'évolution en temps réel des températures des conteneurs réfrigérés à l'aide de courbes fluides et d'un quadrillage repéré.

1.4 4. Équipe Performance & Multithreading

Membres : Azefack, Lauraine

Mission : Traitements asynchrones et synchronisation des données avec l'UI.

Pour répondre à la contrainte de temps réel sans bloquer l'interface, l'équipe a implémenté la classe `MoniteurTemperature.java` qui tourne sur un thread d'arrière-plan (via l'interface `Runnable`). Ce thread se réveille toutes les 3 secondes pour modifier la température des contenueurs réfrigérés. Pour mettre à jour le `GraphiqueTemperature` et le tableau des alertes en toute sécurité, ils ont utilisé `SwingUtilities.invokeLater` :

```
// Dans MoniteurTemperature.java
@Override
public void run() {
    while (actif) {
        try {
            Thread.sleep(3000); // Pause de 3 secondes
            gestionDonnees.appliquerVariationTemperature();

            // Mise a jour de l'IHM (JTable, Graphique) de maniere Thread-Safe
            SwingUtilities.invokeLater(() -> {
                if (onActualisation != null) onActualisation.run();
            });
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
        }
    }
}
```